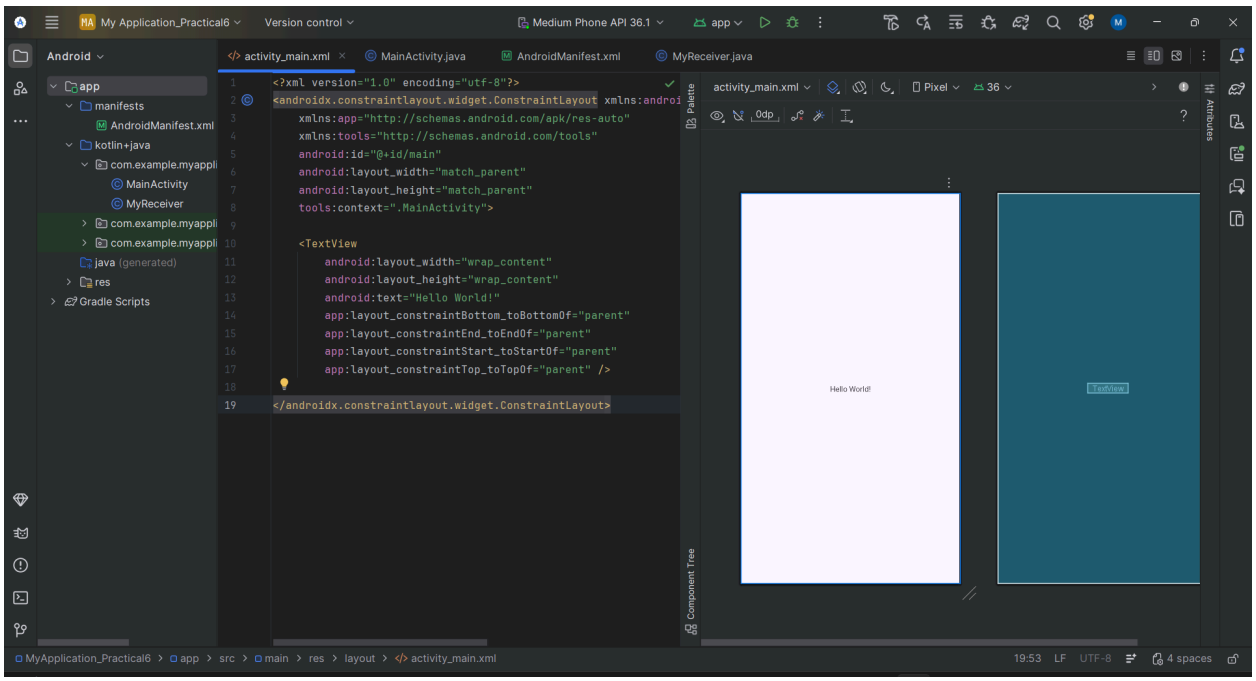
Practical 6

Aim: Implementing broadcast receivers. Design a broadcast receiver to respond to system or application-level broadcasts. Test the receiver with different broadcast scenarios.

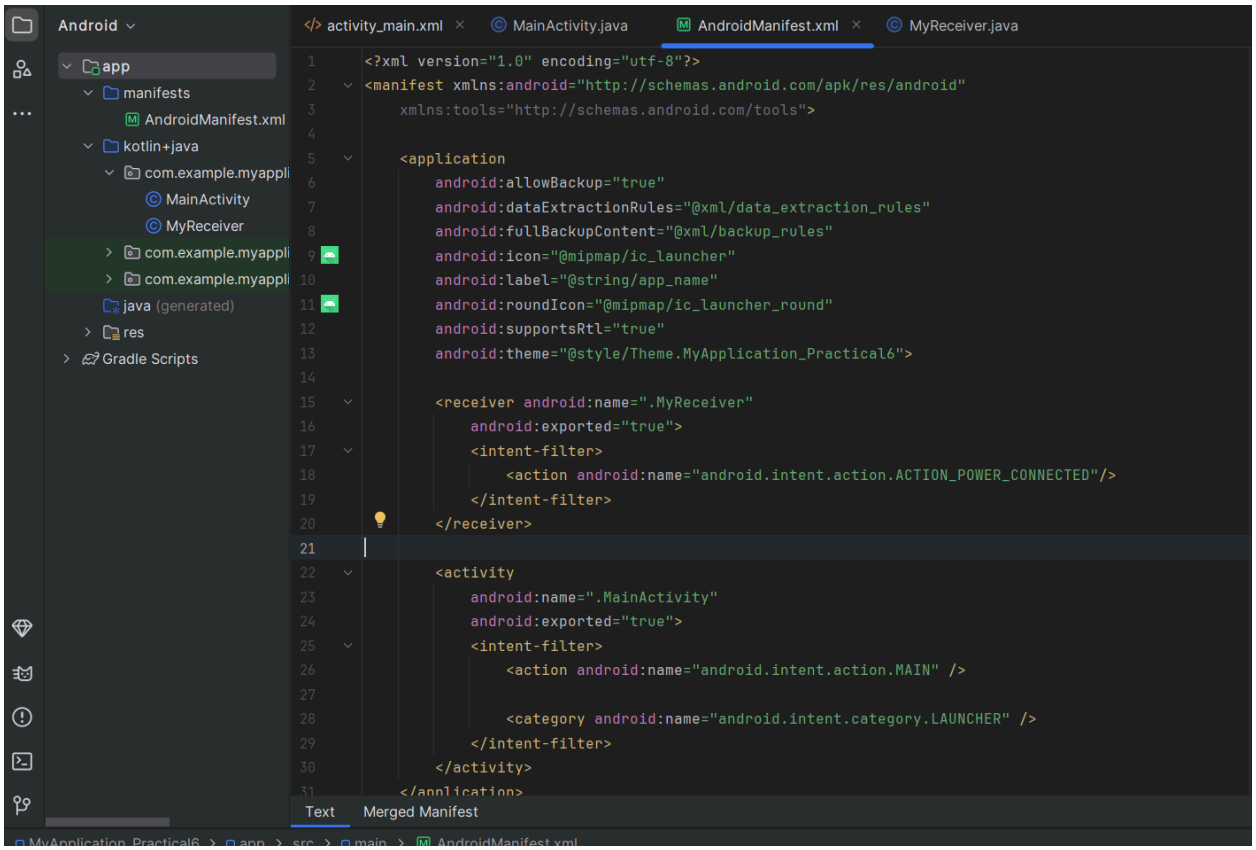
Theory: This practical demonstrates Broadcast Receivers, which listen to system or application-level events. When a specific event, like a power connection, occurs, the onReceive() method is triggered. It shows how applications respond to real-time system broadcasts.

Codes:

activity_main.xml - This layout defines a simple user interface for the application. It may contain a TextView or a basic layout, but it is not directly involved in the Broadcast Receiver's functionality, which runs in the background.



AndroidManifest.xml - Registers the Broadcast Receiver and specifies the type of broadcast it listens to using an intent filter (e.g., power connected/disconnected).



Output:

MyReceiver.java - This class defines a Broadcast Receiver that listens for system events such as power connection. When the event occurs, it triggers the onReceive() method and displays a Toast message.

Code Explanation (MyReceiver.java)

- BroadcastReceiver → Base class for receiving broadcasts
- onReceive() → Called when a broadcast is received
- Intent → Contains broadcast information
- Toast.makeText() → Displays message on screen

MainActivity.java - This activity serves as the entry point of the application. It provides a basic user interface and launches the app, while the Broadcast Receiver runs independently in the background.

Code Explanation

- onCreate() → Initialises the activity and loads the layout
- setContentView() → Sets the UI from activity_main.xml
- No direct interaction with the Broadcast Receiver
- Broadcast events are handled through the manifest

Conclusion

The Broadcast Receiver was successfully implemented to respond to system-level events. It demonstrated how Android applications can react to real-time system broadcasts.